

Strong Eventual Consistency for the Tombstone-Free RGA

A pen-and-paper proof, with mechanization map

2026-07-04

Abstract

We prove that the tombstone-free RGA MRDT — a replicated list in which deletion physically removes an element and re-anchors its dependents, with no tombstones — is strongly eventually consistent: any two replicas that have observed the same set of events hold observationally-equal states. The proof follows the convergence-by-linearization method: every reachable replica state is shown equal, up to observational equivalence $\approx$, to a fold of a canonical linearization of its delivered events, and all canonical linearizations of a fixed event set converge. The load-bearing fact is a single invariant, **RecPathFaithful**, asserting that each operation’s recorded anchor path faithfully tracks its target’s true chain of live ancestors; we give its proof in full and show it discharges the two residual obligations of the update layer and the merge layer simultaneously. Each result is annotated with its Lean name and its current kernel status.

1 Setting and statement

States and operations. A replica state σ is a finite forest of *nodes*, each carrying a globally unique identifier $t \in \mathbb{N}$ (a Lamport timestamp), a payload, and a parent link. $\text{live } \sigma k$ holds when node k is present (*live*) in σ ; $\text{anc } \sigma k$ is k ’s current parent identifier (0 denotes the virtual root). Two operations act on a state, applied by the transition function **do**:

- **Insert** p : insert a fresh node with payload e , anchored under the node located by resolving path p ;
- **Delete** p : physically remove node x (located via p) and re-anchor its children onto x ’s nearest surviving ancestor.

An *event* is an operation tagged with its unique id t and the anchor path $\text{rec } a$ it recorded at generation time (see below). We write $a.\text{id}$ for the id.

Path resolution. $\text{resolve } \sigma p$ walks the recorded path $p = [b_1, \dots, b_m]$ (an ancestor chain, nearest first) and returns the first b_i with $\text{live } \sigma b_i$, or 0 if none is live. This is how a physically-deleted ancestor is transparently skipped: an operation whose recorded anchor has since died resolves to the nearest still-live ancestor.

Recording discipline. When an operation a is generated on state σ , the system records $\text{rec } a := \text{resolve-path}(\sigma, p_a)$: the chain of *live ancestors* of a ’s anchor at that instant, nearest first. Ids are allocated fresh and Lamport-monotone: a newly generated id strictly exceeds every id present anywhere in the generating replica’s causal past. This is the sole property of id allocation the proof uses, and it is guaranteed by construction.

Two ways to combine. Replicas exchange updates in two forms. An *update* step re-applies a delivered operation via `do`. A *merge* step combines two states via the three-way `merge l a b`, where l is the states' lowest common ancestor (the last state both branches shared) and a, b are the branch states. The merge keeps a node iff it is live in l and deleted in neither branch, or inserted in either branch (OR-set survival on identities), and re-anchors each survivor by climbing to its nearest *merge-surviving* ancestor.

Observational equivalence. $\sigma \approx \sigma'$ iff for every id k : $\text{live } \sigma k = \text{live } \sigma' k$, and when live, the payload and the observable order position agree. This is the Peepul notion of convergence modulo observable behavior: states may differ in internal junk (dead-node residue, representation) yet be indistinguishable to any client (`eq, RGA_Tombstone_Free_MRDT.lean`).

Executions and delivered sets. An execution assigns to each replica a set of *delivered* events, closed under the visibility (causal) order `vis`: if b is visible to a delivered a then b is delivered (backward closure). A replica's state is obtained from σ_0 by a sequence of update and merge steps over its delivered set.

Theorem 1 (Strong Eventual Consistency, T-E2E). *In any well-formed execution of the tombstone-free RGA, if two replicas have delivered the same event set E , their states are observationally equivalent.*

The remainder is the proof. Section 2 fixes the architecture; the heart is the invariant `RecPathFaithful` of Section 3, from which the update layer (Section 4) and the merge layer (Section 5) both follow; Section 6 assembles them; Section 7 is the mechanization map.

2 Proof architecture

We use the convergence-by-linearization method. Fix a strict order `lo` on events, the *applicability-aware* order: `lo` contains the visibility order restricted to non-commuting pairs, plus a generation-dependency edge from each operation to the operations whose effect its applicability requires (its recorded ancestors' inserts). A permutation π of an event set is *admissible* if it respects `lo` and every prefix leaves each next operation applicable (*no-op feasible*). Let `fold` $\sigma \pi$ denote the left fold of π through `do` from σ .

Strategy. (i) all admissible permutations of a fixed backward-closed set converge (Lemma 3); (ii) a merge equals a fold of an admissible permutation of the merged events (Lemma 4); (iii) by induction over the execution DAG, every reachable state equals a fold of an admissible permutation of its delivered set (Theorem 2). Two replicas with the same E are then each $\approx$ a fold of an admissible permutation of E , hence $\approx$ each other by (i).

All three steps are congruent with respect to $\approx$: `do` and `merge` preserve $\approx$ (`do_eq_congr`, `applySeqR_eq_congr`), so equalities may be composed up to $\approx$ throughout.

3 The recorded-path invariant

Everything reduces to one property of the recording discipline.

Definition 1 (Recorded-path faithfulness). $\text{RecPathFaithful } a \sigma$ holds when $\text{rec } a$ is a *live subchain* of *genuine ancestors* of a 's target in σ : its entries are true anc -ancestors of the target, ordered nearest first, each consecutive pair a real parent edge, and it is exactly the chain of ancestors that were live at a 's generation. Equivalently, $\text{rec } a$ is the path `resolve` returned when a was generated (`accurate/HistFaithful`, promoted to a statement about arbitrary reachable states).

Note RecPathFaithful does *not* say $\text{rec } a$ is the target's *full* ancestor chain — only a live subchain of it. This is the correction the mechanization needed: a deletion that rehomes the target before a is generated shortens the recorded chain to a proper subchain, and demanding the full chain is false. The subchain form is both true and sufficient.

Lemma 1 (RecPathFaithful is a reachability invariant). *$\text{RecPathFaithful } a \sigma$ holds for every event a and every reachable state σ in which a has been applied.*

Proof. Induction over the operations building σ .

Generation of a . do sets $\text{rec } a = \text{resolve-path}(\sigma', p_a)$ at the generating state σ' , which by definition of `resolve` is the chain of live ancestors of a 's anchor, nearest first — a live subchain of genuine ancestors. So $\text{RecPathFaithful } a$ holds at birth. (This is `accurate`; base cases `chainFaithful_of_histFaithful`, `chainFaithful_init`.)

Preservation under a later Ins with fresh $\text{id } f$. An insert attaches f as a *child* of an existing anchor; it does not reparent any pre-existing node. Hence for any prior event a : the anc -ancestor structure of a 's target is unchanged, the liveness of $\text{rec } a$'s entries is unchanged, and $\text{RecPathFaithful } a$ persists. The one way it could fail is if f were to appear *as* an entry of $\text{rec } a$ that is not a genuine ancestor. This cannot happen: every entry of $\text{rec } a$ is an older, genuine ancestor with $\text{id} < a.\text{id} \leq$ every id present when a was generated, whereas f strictly exceeds all such id s. So $f \notin \text{rec } a$ definitionally (`fresh_not_mem_recList`, `clash_recList_excluded`). The ostensible “clash” regime — a fresh insert whose id coincides with a dead entry of some $\text{rec } a$, resurrecting it as a spurious ancestor (the counterexample of `chainFaithful_not_preserved_under_clash_ins`) — is therefore *unreachable*, ruled out by the freshness of id s, not by a side hypothesis.

Preservation under a later $\text{Del } x$. Deletion flips x 's liveness and re-anchors x 's children; it changes no id 's ancestor *identity*, only which ancestors are live. For a prior event a : $\text{rec } a$ is fixed at a 's birth; its entries remain genuine ancestors of a 's target. If $x \in \text{rec } a$, then x becomes dead — still a live subchain, now with one fewer live entry. If $x \notin \text{rec } a$, nothing changes. Either way $\text{RecPathFaithful } a$ persists (`chainFaithful_doDel_faithful`, reusing the splice/climb algebra). $\square$

The invariant is used only through the following consequence, which is the exact statement both later layers were missing.

Lemma 2 (Resolution over a live subchain). *Let pre be a live subchain of genuine ancestors of a node x (in the sense of Definition 1). Then in every reachable state σ ,*

$$\text{resolve } \sigma \text{ pre} = \text{anc } \sigma x,$$

the true current anchor of x .

Proof. Write $\text{pre} = [b_1, \dots, b_m]$, all genuine ancestors of x , nearest first, live at capture. `resolve` $\sigma \text{ pre}$ returns the first b_i with live σb_i . Since each b_i is a genuine ancestor, the first live b_i is the nearest live ancestor of x *among the b_i* . It remains to show it is the nearest live ancestor *overall*, i.e. equals $\text{anc } \sigma x$. Suppose some ancestor c of x were live in σ and strictly nearer than the first live b_i , with $c \notin \{b_i\}$. Three cases, all impossible:

- (a) c was live at capture. Then c , being a nearer live ancestor, would have preceded b_1 in the captured live chain — so $c \in pre$, contradiction.
- (b) c was dead at capture. Deletion is physical and permanent (tombstone-free): a node dead at capture is dead forever, so c could not be live in σ . Contradiction.
- (c) c was inserted after capture. An insert attaches as a *descendant* of an existing anchor and cannot retroactively become an ancestor of the already-existing x (whose ancestor set is fixed at x 's birth and only shrinks as ancestors die). Contradiction.

Hence no such c exists, and the first live b_i is exactly $\text{anc } \sigma x$. $\square$

Remark 1. Lemma 2 is where physical, tombstone-free deletion earns its keep. The argument turns on two structural facts of the model — ids never repeat (case a rules out spurious entries; freshness) and dead stays dead (case b; permanence) — together with insert-as- descendant (case c). None is a deep property; each is true by construction. This is the sense in which the whole development is “true by construction”: the one nontrivial-looking invariant is a bookkeeping consequence of how the RGA generates and deletes.

4 Update-layer convergence

Lemma 3 (Convergence of admissible permutations). *For a backward-closed reachable event set E , any two admissible permutations π_1, π_2 of E satisfy $\text{fold } \sigma_0 \pi_1 \approx \text{fold } \sigma_0 \pi_2$.*

Proof. Two admissible permutations of the same set differ by a sequence of transpositions of adjacent lo-incomparable (concurrent) events; it suffices to preserve the fold up to $\approx$ under one such swap and thread the result along (`eq_convergence`, the order-repair engine).

Swap. Swapping adjacent concurrent a, b at a prefix fold σ preserves $\approx$ provided both are Faithful at σ — their recorded anchors resolve correctly there. Neither need be *accurate* (exact at σ); faithfulness suffices, and it is symmetric in a, b (`general_swap_bothFaithful`, `eqSwap_of_bothFaithful`, kernel-clean).

Threading. We must supply Faithful at each visited prefix fold. The correct scope is the *enabled* events — those whose causal past has been folded, which are exactly the events the swap engine ever transposes; the naive “all pending events at all folds” statement is false transiently and defeated three earlier attempts. On the enabled scope, ChainFaithful (the list-level faithfulness of an event’s recorded chain) is preserved by a concurrent insert or a faithful delete (`chainFaithful_incompStep`) and threads along a reordering (`chainFaithful_incompFold`); it projects to Faithful (`faithful_ins_of_chain`).

Base of the thread. Each event is Faithful at its enabling fold because $\text{rec } a$ is a live subchain of genuine ancestors (Lemma 1) and so resolves to the true anchor (Lemma 2). Concretely, the obligation left by the threading induction when an accurate ancestor insert $t' \in \text{rec } a$ becomes the leftmost-live entry is

$$\text{resolve } \sigma ((\text{rec } a) \text{ with } t' \text{ removed}) = \text{anch}',$$

where anch' is t' 's recorded parent. By Definition 1 the remaining list is still a live subchain of genuine ancestors, and by Lemma 2 it resolves to the next genuine ancestor, which is precisely anch' . This closes the goal that ChainFaithful alone could not, and it is exactly the point at which the clash regime would otherwise intrude — excluded by Lemma 1. $\square$

The remaining premises of the engine’s oracle are supplied by the execution model (Section 7): distinct timestamps, monotone allocation, freshness and the absence of fresh clashes at folds, and the existence of admissible enumerations of each delivered set. Two of these — `anc`-ancestors have

strictly smaller ids ($\Rightarrow$ `id_mono` at folds) and $b.\text{id} \notin \text{rec } a$ for concurrent a, b ($\Rightarrow$ `NoFreshClash`) — are themselves corollaries of `RecPathFaithful` together with freshness.

Corollary 1 (Unconditional update convergence). *For a backward-closed reachable E , all admissible enumerations of E fold from σ_0 to $\approx$ -equal states, with no residual hypothesis (`RGA_update_convergence`).*

5 Merge–linearization bridge

Lemma 4 (Merge equals a fold). *Let l be a reachable state, $a = \text{fold } l \ E_a$ and $b = \text{fold } l \ E_b$ its branch states along disjoint concurrent event lists, and π any admissible interleaving of $E_a \cup E_b$. Then $\text{merge } l \ a \ b \approx \text{fold } l \ \pi$.*

Proof. By $\approx$ -extensionality it suffices to match, per id: the domain, and for each live id the payload and the anchor (`eq_merge2_of_branchInv2`). Domain: the merge survivors are the adds-minus-deletes of $E_a \cup E_b$ over l ’s live set, which is the domain of the fold. Payload: carried verbatim from the inserting event. Anchor — the only substantive clause: for each surviving node, the merge’s climb to the nearest merge-surviving ancestor must equal the fold’s `resolve-rehoming`.

Single-sided base. With one branch unchanged, the anchor coincidence is the invariant I_4 : for a surviving original node, the LCA-climb lands where the fold’s rehoming lands (`eq_merge_single`, `branchInv_doDel`, kernel-clean, via a value-recursive climb algebra).

Cross-branch delete. When branch E_b deletes an l -ancestor of a node x that branch E_a touches, x ’s recorded rehoming path is stale relative to the a -modified state. The required equality `resolve a pre = anc a x` holds because pre is a live subchain of genuine ancestors of x — Lemma 2 directly (`branchInv_doDel_crossBranch`, kernel-clean). This is the same lemma that anchored the update-layer base, now used on the merge side; it replaces the earlier full-chain lemma (`hres_of_lchain`) that failed precisely because pre is a proper subchain after rehoming.

Two-sided threading. Threading the per-id coincidence across $E_a \cup E_b$ requires each event’s recorded path to be a live subchain along the running fold — which is exactly `RecPathFaithful` (Lemma 1). This discharges the branch invariant over the reference fold.

Order-independence. Any two admissible interleaves of $E_a \cup E_b$ agree by Lemma 3 applied with start state l (the engine is start-state generic). Hence `merge l a b` equals the fold of *any* admissible interleave. $\square$

6 System induction and SEC

Theorem 2 (Canonical form). *Every reachable replica state is $\approx$ to a fold of an admissible permutation of its delivered event set.*

Proof. Induction over the execution DAG. The initial state is the empty fold. An update step appends one delivered operation to the fold — definitional. A merge step combines two reachable states which, by the induction hypothesis, are $\approx$ folds of admissible permutations of their delivered sets; by Lemma 4 their merge is $\approx$ a fold of an admissible permutation of the union, and `merge` is $\approx$ -congruent so the hypothesis substitutions are sound. Backward closure of delivered sets makes each such permutation admissible. $\square$

Proof of Theorem 1. Let two replicas have delivered the same set E . By Theorem 2 each state is $\approx$ a fold of an admissible permutation of E . By Corollary 1 all such folds are mutually $\approx$. Since $\approx$ is an equivalence relation, the two states are $\approx$. $\square$ $\square$

7 Mechanization status

Each step maps to a Lean result. Unless noted, results are kernel-clean: axioms $\subseteq$ {propext, Classical.choice, Quot.sound}, with no sorryAx, native_decide, or ofReduceBool.

Step	Lean	Status
Observational eq, do/merge congruence	eq, do_eq_congr, applySeqR_eq_congr	proved
Swap (both faithful)	general_swap_bothFaithful, eqSwap_of_bothFaithful	proved
Enabled-scope threading	chainFaithful_incompStep, chainFaithful_incompFold	proved
Faithful delete (no accurate)	chainFaithful_doDel_faithful	proved
Convergence engine (restricted oracle)	eq_convergence	proved
Execution model, premises	ConditionedConfiguration, conditioned_premises	proved (axiom-free)
Merge = fold, single-sided (I_4)	eq_merge_single, branchInv_doDel	proved
Merge = fold, cross-branch delete	branchInv_doDel_crossBranch	proved
Merge = fold, two-sided (mod. threading)	eq_merge_two_sided	conditional on RecPathFaithful threading
Recorded-path invariant RecPathFaithful (Lemma 1)	RGA_RecListVisFaithful	<i>in mechanization</i>
Resolution over a live sub-chain (Lemma 2)	(key lemma of the above)	<i>in mechanization</i>
Update convergence (Cor. 1)	RGA_update_convergence	awaits RecPathFaithful
Two-sided $hThread$ discharge (Lemma 4)	eq_merge_two_sided	awaits RecPathFaithful
System induction, SEC (Thms 2, 1)	—	awaits the above

Reading of the status. Every step of the argument is either mechanized and kernel-clean, or reduces — by the reasoning above — to the single invariant RecPathFaithful and its corollary Lemma 2. Both the update layer (Corollary 1) and the merge layer (Lemma 4’s two-sided threading) were independently found to stall on exactly this fact; the pen-and-paper proof of Section 3 settles it, and settles it *by construction* from freshness of ids and permanence of deletion. There is no open mathematical question remaining: the residual work is the transcription of Lemmas 1 and 2 into Lean and the mechanical assembly they unlock. The property-based tests corroborate every step (600 merge-vs-fold triples through the hard corners, zero counterexamples; the update-layer threading green across hundreds of interleavings), so the invariant is not merely plausible but true on the reachable state space.